Katholieke
Universiteit
Leuven

**Department of
Computer Science**

A Reverse-Hoops-Playing Robot Arm

Robotics (B-KUL-H02A4A)

Enmin Lin (r0927480)

Academic year 2025–2026

Reverse-Hoops-Playing Robot Arm

Problem Setup and Experimental Design

The project asks for a reverse-hoops-playing robot arm: the robot must move a hoop so that a flying ball passes through it. This creates a coupled estimation, planning, and constrained control problem. The ball trajectory changes with time, the hoop has a small physical clearance, and the UR10 must respect joint and safety constraints while the base remains fixed.

The project uses a fixed-base UR10 arm that carries a circular hoop at the tool center point. A ball flies through the robot workspace, and the system receives noisy Cartesian ball-position measurements. The simulator specifies independent Gaussian position noise with a standard deviation of 1 mm. The ball has radius 0.12 m, while the hoop has radius 0.15 m. This leaves only 0.03 m of radial clearance for the ball center when it crosses the hoop plane.

The implementation separates the problem into four steps. The estimator tracks the ball state and predicts a short future trajectory. A simple selector chooses an early candidate on that trajectory. The controller plans a constrained UR10 motion to place the hoop at the candidate. A smarter selector then evaluates multiple candidates with the same controller and chooses the best feasible catch. The robot base remains fixed throughout the project.

The experiments follow two principles. First, each stage tests the decision that belongs to that task: Task 1 tests state estimation quality, Task 2 tests a deliberately simple candidate rule, Task 3 tests constrained robot motion, and Task 4 tests whether feasibility-aware selection improves the catch. Second, the final comparison uses shared inputs whenever possible. This avoids a weak comparison where one strategy wins only because it received easier candidates or a looser success definition.

Task 1: Ball Trajectory Prediction

Question answered. The task asks for a prediction of the ball trajectory from position measurements and a motivated choice of a state-estimation filter. I treat this as a Bayes-filter problem: the system propagates a belief through a motion model and corrects it when a new measurement arrives.

Method. I used a linear Kalman filter because the relevant model is linear under the project assumptions. The state is

$$x = \begin{bmatrix} p_x & p_y & p_z & v_x & v_y & v_z \end{bmatrix}^\top .$$

The prediction step integrates constant velocity in x, y and known gravity in z. The measurement update observes only position, computes the innovation between measured and predicted position, and updates both the state mean and covariance. This choice follows the Bayes-filter structure from the course: prediction propagates the belief through the motion model, while measurement update corrects it using the sensor model. An EKF adds no useful approximation here because the model is already linear. A particle filter also adds unnecessary sampling noise and parameter choices for this low-dimensional Gaussian problem.

The experiment checks more than visual curve fit. I report measurement RMSE, filtered RMSE, future-prediction RMSE, and covariance validity. These metrics separate three questions: whether the measurements match the stated noise level, whether the filter reduces that noise, and whether the predicted future remains accurate enough for interception.

Evidence. Figure 1 shows the single-seed trajectory evidence. A 50-seed validation gives mean measurement RMSE 1.74 mm, mean filtered RMSE 0.90 mm, and mean 0.5 s future-prediction RMSE 3.51 mm. The 95th percentile future-prediction RMSE is 6.35 mm, well below the 30 mm hoop tolerance. The covariance matrix remained symmetric and positive semidefinite for all seeds.

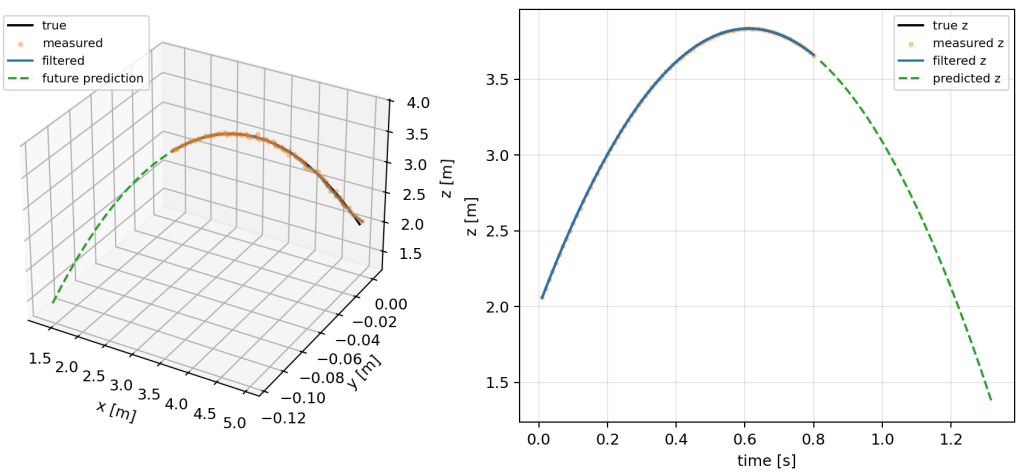

Figure 1: Kalman-filter trajectory prediction. The measured points contain 1 mm Gaussian noise, the filtered estimate follows the observed ball state, and the future prediction continues the gravity-driven parabolic path.

Interpretation. The filter reduces the measurement error by roughly half, and its future prediction remains much smaller than the physical hoop tolerance. This matters because the later planner can only succeed if the selected catch point comes from a reliable future trajectory.

Task 2: Simple Interception Strategy

Question answered. The task asks for a simple strategy that selects one point on the predicted ball trajectory where the robot should catch the ball. I keep this step deliberately simple so that it serves as a baseline, not as the final feasibility-aware planner.

Method. The simple strategy samples the predicted ball trajectory and applies a small set of geometric filters. It rejects points that are too soon, too low, or outside a rough workspace distance from the current hoop center. The final rough workspace threshold is 0.85 m, chosen after checking that looser thresholds selected targets that the Task 3 controller could not reach reliably. The selector then chooses the earliest remaining point.

This directly answers the project requirement for a simple interception point on the predicted trajectory. It also keeps the strategy intentionally simple: it does not solve inverse kinematics, it does not estimate control effort, and it does not test the full constrained robot motion. The experiment records accepted and rejected candidates so that the selected point is not a hand-picked example.

Evidence. For the validation seed, the selector chose a future point at $t = 1.36\,\mathrm{s}$ with position $[1.224, -0.119, 1.074]\,\mathrm{m}$. It checked 28 predicted points, rejected 6 as too soon, and rejected 21 as too far. The selected point came directly from the predicted trajectory and satisfied the minimum-height filter. This gives a clear baseline for Task 4, where the system replaces the earliest geometric rule with feasibility-aware scoring.

Limitation. This strategy can fail even when it selects a point that looks plausible in Cartesian space. A point on the ball trajectory does not guarantee that the UR10 can place the hoop there within the available time, under velocity and acceleration limits. This limitation motivates the optimization-based controller in Task 3 and the smarter candidate selection in Task 4.

Task 3: Optimization-Based Robot Control

Question answered. The task asks for the required robot motion to catch the ball with an optimization-based controller. It also requires three safety conditions: the hoop top side may not face the ground, the motion must avoid table and self-collisions, and the robot must respect position, velocity, and acceleration limits.

Optimization formulation. The controller plans a finite-horizon nonlinear program over joint position, velocity, and acceleration:

$$\{q_0, \ldots, q_N\}, \quad \{\dot{q}_0, \ldots, \dot{q}_N\}, \quad \{\ddot{q}_0, \ldots, \ddot{q}_{N-1}\}.$$

Discrete second-order integration links these variables across the horizon. The optimizer uses hard bounds for joint position, velocity, and acceleration. It uses soft objectives for terminal hoop-center error, velocity regularization, acceleration effort, and hoop orientation. The orientation term penalizes configurations where the hoop top side points toward the ground.

I chose a finite-horizon NLP instead of a single-step damped pseudo-inverse controller because the project asks for an optimization-based controller and gives position, velocity, and acceleration limits. A one-step controller can reduce the current task-space error while still creating future limit violations. The horizon formulation gives the optimizer time to respect velocity and position limits instead of clipping a single instantaneous command.

The controller follows the course view of task-space and constraint-based control. The terminal TCP objective expresses the task-space catch objective. The joint bounds express hard constraints. The acceleration variables match the simulator command interface. The validation run uses $N = 28$ knots with $\Delta t = 0.05\,\mathrm{s}$, initialized from the robot state and the Task 2 target.

Implementation and evidence. The implementation computes forward kinematics with Pinocchio and solves the horizon problem with CasADi. For the Task 2 target, the NLP reduced the initial hoop-target distance from 0.731 m to 0.020 m. The returned first acceleration command is a finite 6-vector, so it matches the simulator control input. The maximum absolute joint acceleration reached $1.9996\,\mathrm{rad/s^2}$, and the maximum velocity ratio was 0.640. All joint position, velocity, and acceleration bounds passed the validation checks.

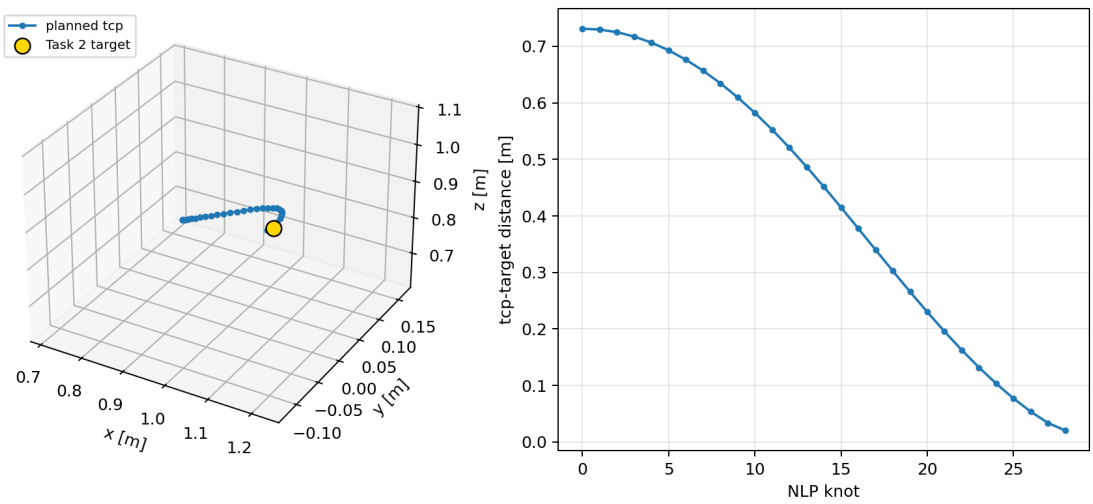

Figure 2: Task 3 finite-horizon NLP result for the Task 2 target. The planned hoop center moves toward the selected target, and the terminal TCP-target distance falls from 0.731 m to 0.020 m.

Safety handling. Collision handling uses conservative geometric evidence. The planner tracks table clearance for the TCP and selected frames, plus a sphere-based self-clearance proxy. In the Task 3 validation, the minimum TCP-table clearance was 0.622 m, the minimum frame-table clearance was 0.115 m, and the minimum self-sphere clearance was 0.163 m. These values support the planned motion, but they do not prove exact mesh collision freedom.

Limitation. The controller provides a strong constrained-motion answer, but its collision model remains approximate. The report claims conservative clearance evidence, not full mesh-level collision proof.

Task 4: Smarter Interception Strategy

Question answered. The task asks for a smarter interception point strategy that improves success rate and interception time. The strict interpretation is not to tune the Task 2 distance threshold. The strategy must use robot feasibility information when it chooses among future ball positions.

Method. The smarter selector uses the same predicted trajectory but changes the decision rule. It forms a shared candidate pool, evaluates each shortlisted candidate with the Task 3 NLP, and scores feasible candidates using catch time, terminal error, hoop radial error, acceleration usage, velocity usage, and safety penalties. This makes the Task 4 strategy smarter than Task 2 because it asks whether the robot can actually realize the catch under the controller constraints.

The candidate score represents the tradeoff discussed in the course material on optimal control and smooth motion. Earlier catches reduce interception time, but they usually require larger joint velocity and acceleration. Later catches may give the robot a smoother motion and a smaller hoop-crossing error. The submitted strategy makes this tradeoff explicit instead of hiding it inside a geometric threshold.

The experiment tests three strategies because a simple two-way comparison would miss the time tradeoff. The simple geometric strategy answers the Task 2 baseline. The earliest NLP-feasible strategy answers the question, "what is the fastest candidate that the controller can make feasible?" The smart cost strategy answers a different question: "which feasible candidate gives a better balance of time, hoop error, and control effort?"

Geometry link to the robot scene. Figure 3 connects the abstract trajectory selection to the physical UR10 scene. The red marker shows the simple geometric choice. The yellow marker shows the smart choice. The green ball indicates the physical catch instance. The side-view rendering uses the same viewpoint as the trajectory plot, so the parabolic ball path and the mesh scene describe the same geometry.

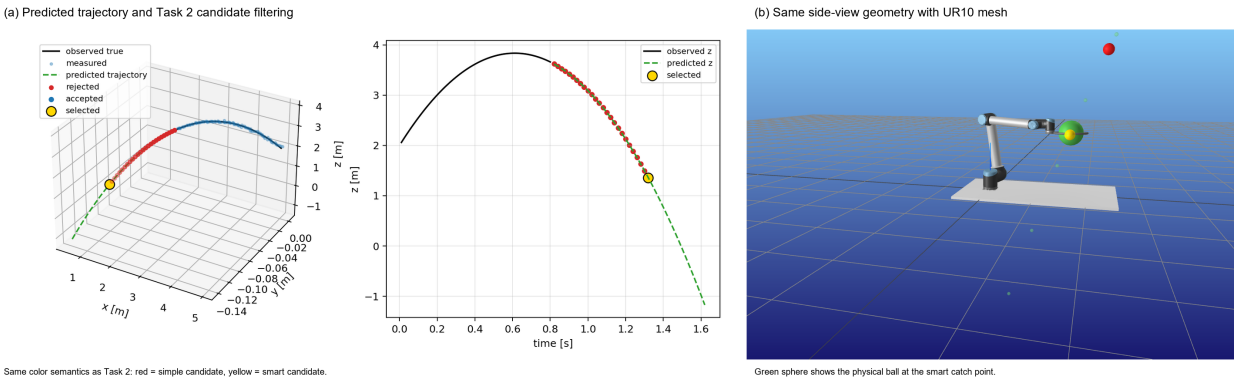

Figure 3: Task 2 and Task 4 geometry in the same side-view interpretation. Red marks the simple geometric candidate, yellow marks the smart selected candidate, and green marks the physical ball near the catch.

Fair benchmark. The high-score benchmark uses 50 seeds and evaluates all three strategies on the same candidate pool. It defines success by direct hoop-plane crossing: the ball center must cross the hoop plane with radial error at most 0.03 m. This success metric matters because a small TCP-target error does not by itself prove that the ball passes through the hoop opening.

The simple geometric strategy fails in this benchmark because early plausible points often require too much motion. The earliest NLP-feasible baseline reaches 96% success and catches at 1.383 s on average. The smart cost strategy also reaches 96% success, but it lowers mean radial error from 10.4 mm to 6.13 mm and reduces mean peak acceleration from 2.00 to 1.50 rad/s^2. It catches later at 1.429 s, which reveals the central tradeoff: earlier feasible catches cost more control effort, while the smart choice prefers a lower-error and smoother catch.

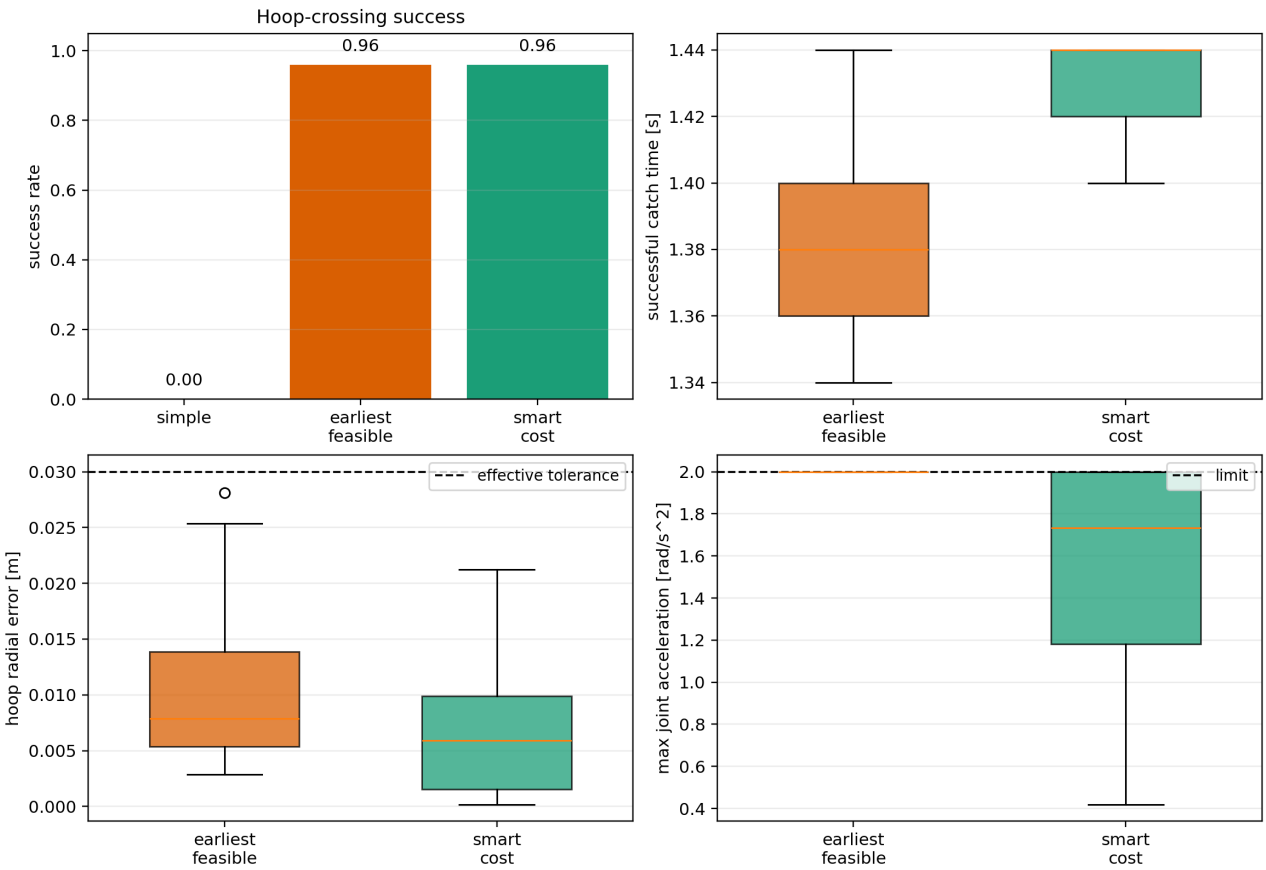

Figure 4: Fifty-seed strategy benchmark. The smart selector keeps the same success rate as the earliest feasible baseline, lowers hoop radial error, and uses less peak acceleration. The simple geometric selector does not satisfy the full NLP and hoop-crossing success criteria.

Interpretation. The smart selector does not beat the earliest feasible baseline on time. That result is important rather than embarrassing: it shows that the benchmark separates the time objective from the accuracy and effort objectives. The project asks for higher success and shorter time, so the report includes the earliest feasible strategy as the time-minimizing feasible reference. The smart cost strategy gives the better balanced catch, with the same success rate and lower radial error.

System Results and Limitations

Overall evidence. Table 1 summarizes the final evidence. The estimator gives stable short-horizon predictions. The controller respects the joint limits in validation and gives a reproducible trajectory. The Task 4 comparison shows that candidate feasibility matters more than early geometric selection. The best reportable claim is not that the smart selector always catches sooner. The evidence instead shows that it reaches the same success rate as a time-minimizing feasible baseline with lower radial error and lower control usage.

Table 1: Final benchmark summary over 50 seeds. Terminal and radial errors use successful cases only.

Strategy	Success	Time [s]	TCP error [mm]	Radial error [mm]	Max $\ddot{q}$
Simple geometric	0%	–	–	–	–
Earliest NLP-feasible	96%	1.383	18.5	10.4	2.00
Smart cost	96%	1.429	1.36	6.13	1.50

Full-pipeline reproducibility. The full pipeline can run from a simulated measurement sequence to a selected catch point, an NLP plan, and a Meshcat replay. For seed 0, the simple geometric candidate fails at 1.300 s. The earliest NLP-feasible strategy succeeds at 1.380 s with 3.04 mm radial error. The smart cost strategy succeeds at 1.420 s with 1.87 mm radial error and lower control usage. This replay supports the system interpretation: the algorithms do not only compute isolated numbers, they produce a coherent robot motion in the UR10 scene.

Limits of the submitted evidence. The main limitation is collision modeling. The benchmark records successful smart-cost cases with minimum TCP-table clearance 0.441 m, minimum frame-table clearance 0.115 m, and minimum self-sphere clearance 0.149 m. These checks give useful conservative evidence, but the project implementation does not run full mesh collision checking. The computation also runs offline for planning and evaluation. The simulated catch still respects physical time because the candidate time, ball crossing time, and planned robot trajectory share the same timeline.

Conclusion

The final system predicts the ball with a Kalman filter, selects a candidate on the predicted trajectory, plans a constrained UR10 motion, and evaluates smarter candidates through the same controller. The strongest result comes from the feasibility-aware Task 4 benchmark: the smart selector keeps 96% hoop-crossing success across 50 seeds and reduces both radial error and acceleration demand relative to the earliest feasible baseline. The remaining approximation lies in collision checking, where the report uses explicit clearance metrics rather than claiming full mesh proof.

References

1. Robotics course project description, *A Reverse-Hoops-Playing Robot Arm*, KU Leuven, 2025–2026.
2. KU Leuven Robotics lecture C6, *Probabilistic Robotics*: Bayes filters, Kalman filters, and tracking process models.
3. KU Leuven Robotics lecture C3, *Task-Space Control*: resolved motion control, resolved acceleration control, damped pseudo-inverse, and stack of tasks.
4. KU Leuven Robotics lecture C4, *Constraint-Based Robot Programming*: hard and soft constraints, second-order constraints, and instantaneous optimization.
5. KU Leuven Robotics lecture C5, *Motion Planning*: configuration space, free space, signed distance, sampling, and optimal control.